



SSC502 – Laboratório de ICC I

Introdução à C

Prof.Dr. Danilo Spatti

- O C nasceu na década de 70. Seu inventor, **Dennis Ritchie**, implementou-o pela primeira vez usando um DEC **PDP-11** rodando o sistema operacional **UNIX**.
- O C é derivado da linguagem **B**, criada por Ken Thompson.
- O B, por sua vez, veio da linguagem **BCPL**, inventada por Martin Richards.

- Permite a **manipulação** de bits, bytes e endereços.
- Trabalha com **4** tipos de dados.
- O C ANSI é composto por **32** palavras **reservadas**.
- Linguagem do tipo **estruturada**.

asm: Indica instruções em assembly.

auto: Define a classe de armazenamento padrão, ou seja , a variável é criada quando a função é invocada e depois é destruída quando a função termina.

break: Comando para forçar a saída imediata dos comandos switch, for, while, e , do...while.

case: Utilizado dentro do comando switch para selecionar uma constante.

char: Tipo de dados utilizado para armazenar caracteres.

const: Impede que uma variável seja modificada, criando assim uma constante.

continue: Força a interrupção dos loops for , while , ou do...while fazendo com que passem para a próxima iteração.

default: É utilizado dentro de switch...case para tratar valores não definidos anteriormente nas opções case.

do: Estrutura de repetição usada em conjunto com o while . O comando do...while faz com que os comandos do bloco a ser repetido sejam executados no mínimo uma vez.

double: Tipo de dados numérico de ponto flutuante com precisão dupla.

else: Indica um bloco de comandos a ser executado quando a condição do comando if for falsa.

enum: Tipo de dados definido pelo programador que permite a definição de constantes

extern: Indica que uma variável foi declarada em outro arquivo.

float: Tipo de dados de ponto flutuante com precisão simples.

for: Estrutura de repetição que utiliza condições e contador.

goto: Comando que permite “saltar” incondicionalmente para uma posição do programa pré-determinada.

if: Comando condicional que altera o fluxo do programa de acordo com uma condição que pode ser verdadeira ou falsa.

int: É o tipo de dados mais usado para armazenar valores inteiros.

long: É um tipo de dados usado para armazenar valores inteiros que possui precisão superior ao tipo int..

register: Solicita o armazenamento diretamente nos registradores do processador.

return: Retorna um valor dentro de uma função ou força o abandono da mesma.

short: É o chamado inteiro curto. Armazena valores inteiros com precisão menor (apenas 2 bytes) do que o tipo int.

signed: Sinaliza que uma variável pode armazenar tanto valores positivos como negativos.

sizeof: Comando usado para retornar o tamanho em bytes de um determinado tipo de dados.

static: Comando que avisa ao compilador para reter o valor de uma determinada variável.

struct: Cria as chamadas estruturas ou seja registros.

switch: Comando de seleção usando em conjunto com o comando case, permite escolher entre várias opções.

typedef: É um comando modificador usado para criar novos nomes para tipos de dados.

union: Permite que um conjunto de variáveis partilhem o mesmo espaço na memória.

unsigned: Faz com que a variável apenas aceite valores positivos ou zero.

void: Comando que indica que a função não retorna nada ou que não tem parâmetros de entrada.

volatile: Indica que uma variável pode ser alterada.

while: Estrutura de repetição que executa enquanto uma condição é verdadeira.

Programa Principal

{

Funcao1

{

Sequencia de Comandos

}

Funcao2

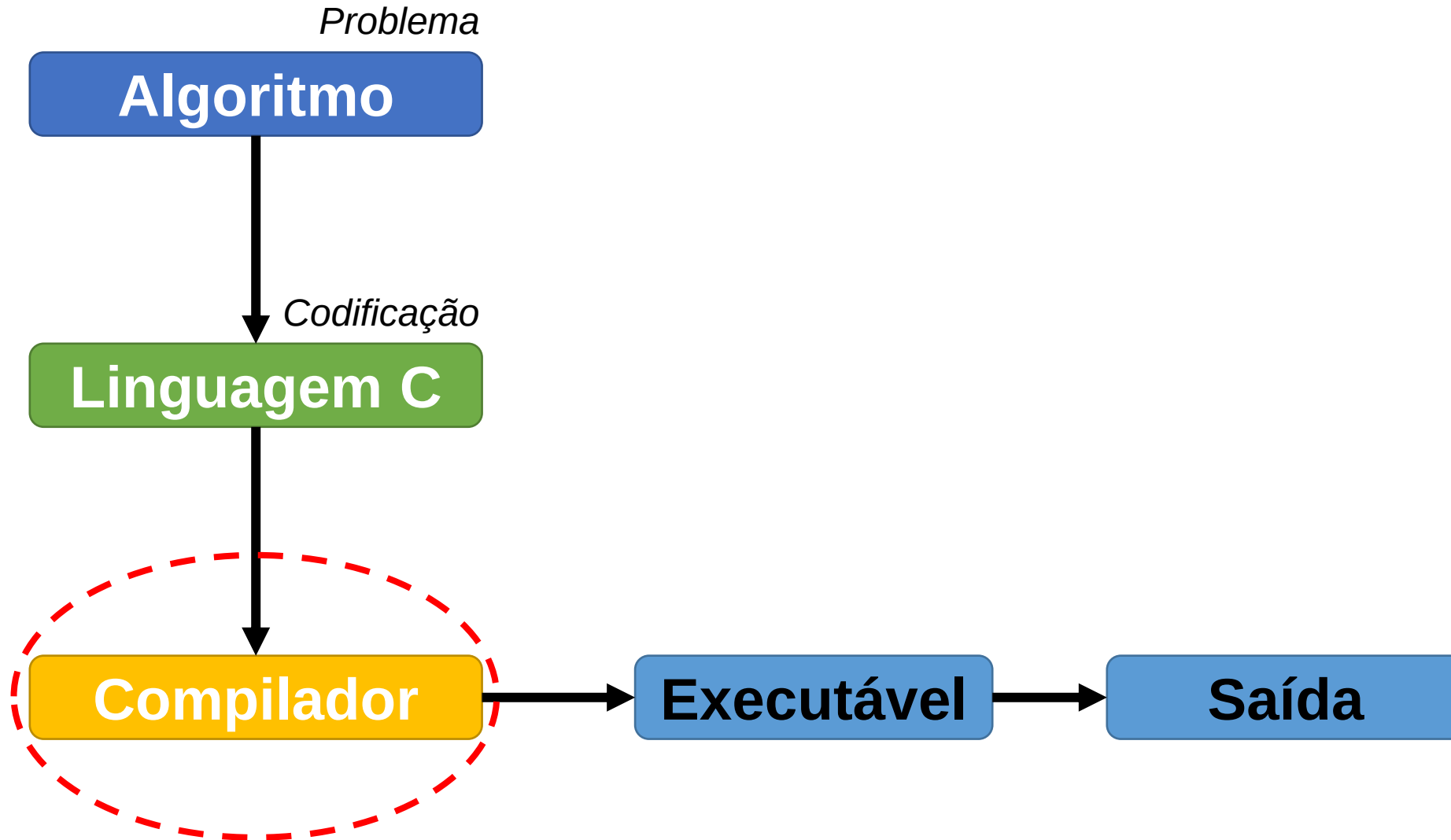
{

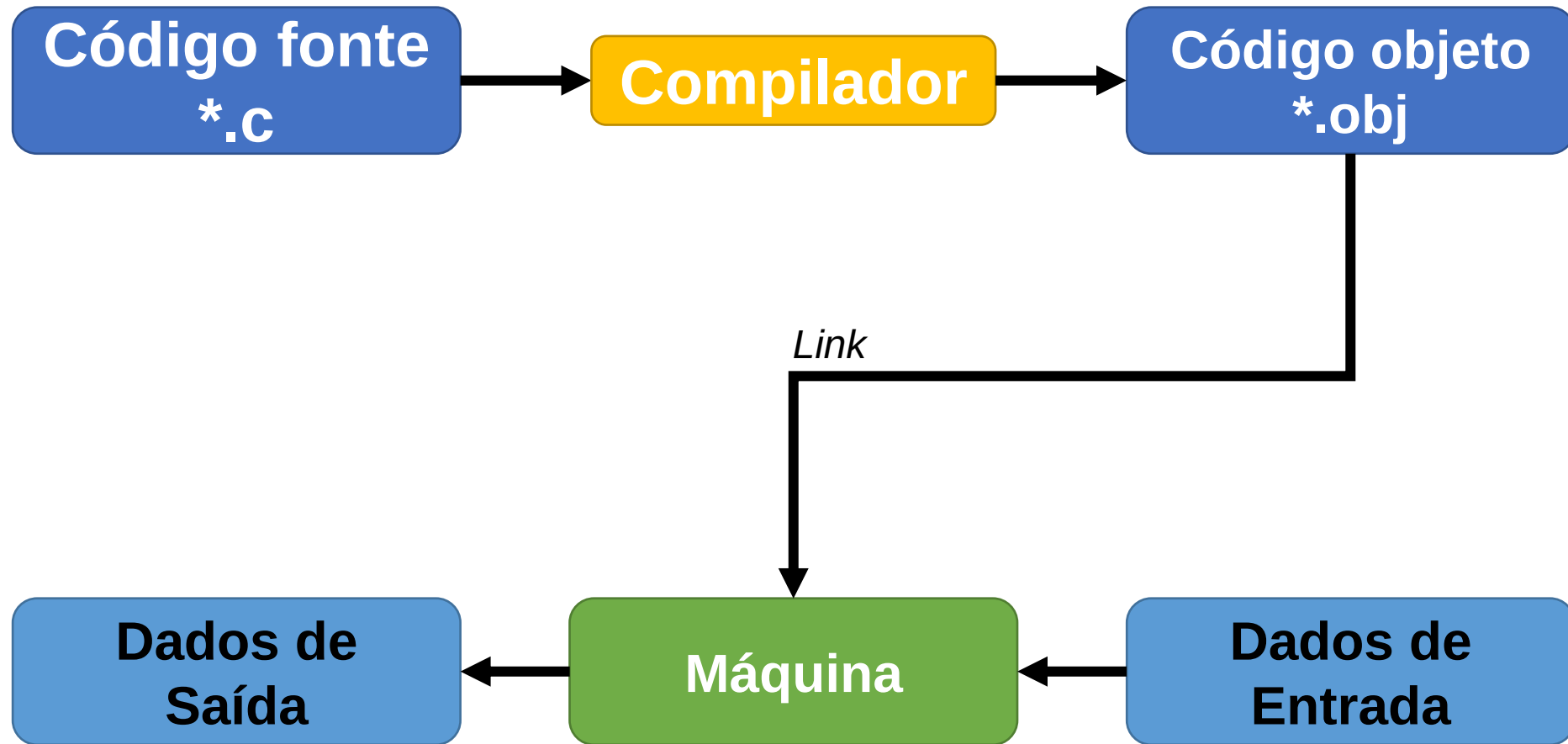
Sequencia de Comandos

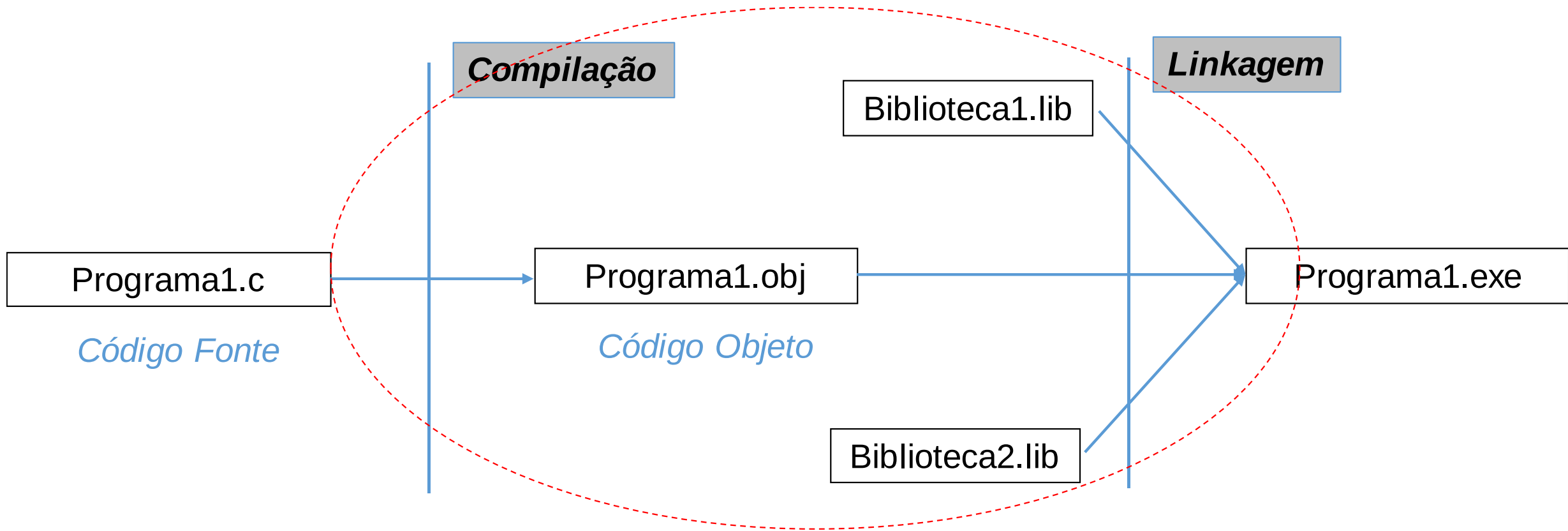
}

}

- Se declarar uma variável com o nome **soma** ela será diferente de **Soma**, **SOMA**, **SoMa** ou **sOmA**.
- Da mesma maneira, os comandos do C **if** e **for**, por exemplo, só podem ser escritos em **minúsculas** pois senão o compilador **não** irá interpretá-los como sendo **comandos**, mas sim como variáveis.







- Estrutura do código *.c:
 - Comentários
 - Diretivas para o compilador
 - Procedimento principal

<inclusão de bibliotecas>

<declaração de dados globais>

<declaração dos protótipos das funções>

```
int main()  
{  
    <corpo do programa principal>  
    return 0;  
}
```

Sempre composto por uma função principal

<funções do programa>

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

Comentário

Diretivas para o compilador

Procedimento principal

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

Raca Caaso

Process returned 0 (0x0) execution time : 0.000 s
Press any key to continue.

- Texto que é ignorado pelo compilador.
- Útil para descrever trechos do algoritmo.
- Possível em qualquer porção do programa.
- Uma linha: `// comentário`
- Várias linhas: `/* comentário
...comentários...*/`

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```


- Colocadas no **início** do código fonte.
- Definem para o compilador as **funções** que serão utilizadas.

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

```
#include<stdio.h>
```

- A linha
`#include <stdio.h>`
diz ao compilador que ele
deve **incluir** o
arquivo-cabeçalho **stdio.h**.
- Neste arquivo existem **declarações** de funções **úteis** para **entrada** e **saída** de dados.
- Toda vez que se quiser **usar** uma destas **funções** deve-se **incluir** este comando.
- O C possui **diversos** Arquivos-cabeçalho.

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

- Sequência de **instruções**.
- Pontuação: **ponto-e-vírgula** sinaliza o **final** da instrução.
- Chaves: indica o **início** e o **fim** do bloco de instruções.

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

- A linha `int main()` indica que estamos **definindo** uma **função** de nome `main`.
- Todos os programas em C **têm** que ter uma função **main**, pois é esta **função** que será **chamada** quando o programa for **executado**.
- O conteúdo da **função** é **delimitado** por chaves `{ }`.

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

- A palavra `int` indica que esta função retorna um **inteiro**.
- A última linha do programa, `return 0;`, indica o número **inteiro** que está sendo **retornado** pela função, no caso o número 0.

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

- A única **operação** que o programa realmente faz é **chamar** a função `printf()`, passando a string `"Raca Caaso\n"` como **argumento**.
- É por **causa** do uso da **função** `printf()` pelo programa que devemos incluir o arquivo- cabeçalho **`stdio.h`**.
- A função `printf()` neste caso irá apenas **colocar** a **string** na tela do computador.

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

- O `\n` é uma **constante** chamada de constante **barra invertida**.
- No caso, o `\n` é a constante barra invertida de "new line" e ele é **interpretado** como um comando de **mudança de linha**, isto é, após imprimir **Raca Caaso**, o cursor passará para a próxima linha.

```
1 //Primeiro programa em C
2 #include<stdio.h>
3
4 //Programa para imprimir frase na tela
5
6 int main()
7 {
8     printf("Raca Caaso\n");
9     return 0;
10 }
```

- Programa começa a execução no “main”
- Palavras chaves em **minúsculo**.
- Declarações finalizadas com “;”
- Texto entre **aspas duplas**.
- C é case sensitive.
- A sequência “\n” coloca o cursor no início da próxima linha.
- `printf()` pode ser usado para exibir texto na tela.
- As **chaves** indicam o início/fim de blocos de comandos.

- Inteiro (4, -8, 12)
 - `int`
- Ponto flutuante (0.12, 0.75)
 - `float`
- Precisão dupla (312E+8, 2E-8)
 - `double`
- Caractere ('A', 'b', '\$', '3')
 - `char`

Tipo	Descrição	Bits	Bytes	Faixa de Valores
char	caracteres ASCII de 8 bits	8	1	-128 a 127
int	inteiros	16	2	-32768 a 32767
float	tipo ponto flutuante (real)	32	4	Seis casas decimais
double	tipo ponto flutuante (real) de dupla precisão	64	8	Dez casas decimais

- Cálculo da média de dois números

```
1  #include <stdio.h>
2  int main()
3  {
4      float N1,N2,M;
5      N1 = 7;
6      N2 = 6;
7      M = (N1+N2)/2;
8      printf("Media = %f", M);
9      return 0;
10 }
```

- Cálculo da média de dois números

```
1  #include <stdio.h>
2  int main()
3  {
4      float M,
5          N1 = 7,
6          N2 = 6;
7      M = (N1 + N2) / 2;
8      printf("Media = %f", M);
9      return 0;
10 }
```

```
1  #include <stdio.h>
2  int main()
3  {
4      int Dias;
5      float Anos;
6      printf("Entre com o numero de dias: ");
7      scanf("%d", &Dias);
8      Anos = Dias / 365.25;
9      printf("\n\n%d dias equivalem a %f anos.\n", Dias, Anos);
10     return 0;
11 }
```

- São declaradas **duas** variáveis chamadas `Dias` e `Anos`.
- A primeira é um `int` (inteiro) e a segunda um `float` (ponto flutuante).
- As variáveis declaradas como **ponto flutuante** existem para armazenar números que possuem casas **decimais**, como 5.1497.
- É feita então uma **chamada** à função `printf()`, que coloca uma **mensagem** na tela.

```
1  #include <stdio.h>
2  int main()
3  = {
4      int Dias;
5      float Anos;
6      printf("Entre com o numero de dias: ");
7      scanf("%d",&Dias);
8      Anos = Dias / 365.25;
9      printf("\n\n%d dias equivalem a %f anos.\n",Dias,Anos);
10     return 0;
11 }
```

- Queremos agora **ler** um dado que será **fornecido** pelo **usuário** e colocá-lo na variável **inteira** `Dias`.
- Para tanto usamos a função `scanf()`.
- A string `"%d"` diz à função que iremos **ler** um **inteiro**.
- O **segundo** parâmetro **passado** à função diz que o dado lido deverá ser **armazenado** na variável `Dias`.

```
1  #include <stdio.h>
2  int main()
3  {
4      int Dias;
5      float Anos;
6      printf("Entre com o numero de dias: ");
7      scanf("%d",&Dias);
8      Anos = Dias / 365.25;
9      printf("\n\n%d dias equivalem a %f anos.\n",Dias,Anos);
10     return 0;
11 }
```

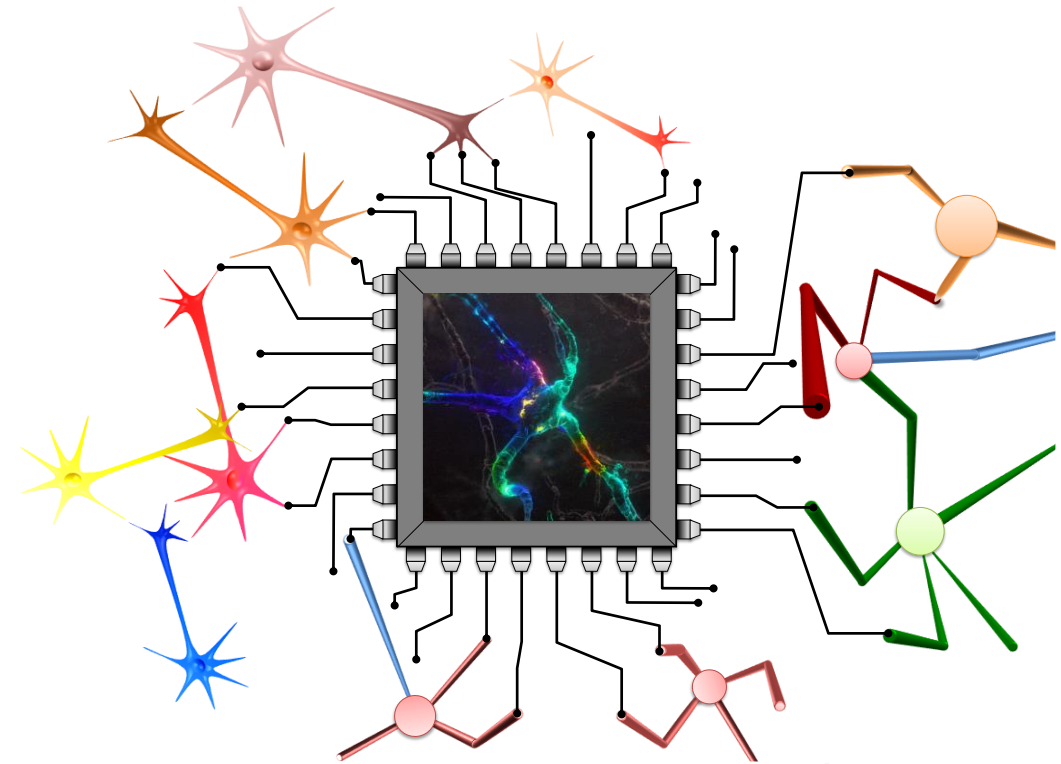
- É importante **ressaltar** a **necessidade** de se colocar um **&** **antes** do nome da variável a ser **lida** quando se usa a função `scanf()`.
- Observe que, no C, quando **temos mais** de **um parâmetro** para uma **função**, eles serão separados por **vírgula**.
- Temos uma expressão **matemática** simples que atribui a `Anos` o valor de `Dias` dividido por `365.25` (`365.25` é uma **constante** ponto flutuante `365,25`).

```
1  #include <stdio.h>
2  int main()
3  {
4      int Dias;
5      float Anos;
6      printf("Entre com o numero de dias: ");
7      scanf("%d",&Dias);
8      Anos = Dias / 365.25;
9      printf("\n\n%d dias equivalem a %f anos.\n",Dias,Anos);
10     return 0;
11 }
```


- Como Anos é uma float o **compilador** fará uma **conversão** automática entre os **tipos** das variáveis.
- A segunda chamada à função `printf()` tem **três** argumentos.
- A string `"\n\n%d dias equivalem a %f anos.\n"` diz à **função** para pular duas linhas, **colocar** um inteiro na **tela**, colocar a mensagem **"dias equivalem a "**, colocar um valor float na tela, colocar a mensagem **"anos."** e pular outra linha.
- Os outros **parâmetros** são as variáveis, `Dias` e `Anos`, das quais devem ser lidos os valores do inteiro e do float, respectivamente.

```
1  #include <stdio.h>
2  int main()
3  ={
4      int Dias;
5      float Anos;
6      printf("Entre com o numero de dias: ");
7      scanf("%d",&Dias);
8      Anos = Dias / 365.25;
9      printf("\n\n%d dias equivalem a %f anos.\n",Dias,Anos);
10     return 0;
11 }
```

spatti@icmc.usp.br



GE4Bio – Grupo de Estudos em Sinais Biológicos